

EcoSearch: A Retrieval-First Framework for Cost-Efficient Question Answering with Minimal LLM Dependence

Author: Maurizio Scibilia

Date: July 2025

Abstract

This paper presents **EcoSearch**, a modular and efficient framework for Retrieval-Augmented Generation (RAG) that reduces runtime reliance on large language model (LLM) calls while maintaining accurate and cost-effective question answering. EcoSearch uses LLMs selectively during preprocessing to generate summaries and guiding questions for each chunk of text. These questions are embedded and indexed with FAISS for fast semantic retrieval. At inference time, user queries are matched against the indexed questions to retrieve relevant chunks. The LLM is invoked only when necessary—for example, when no relevant chunk is found or when answer synthesis requires additional context. This design achieves substantial reductions in LLM usage, latency, and cost while supporting transparent and scalable access to proprietary or internal knowledge sources.

1 Introduction

The widespread adoption of large language models (LLMs) has revolutionized tasks such as question answering, document summarization, and information retrieval. However, this transformation comes at a steep cost: LLM services are expensive, often slow due to remote execution, and raise significant concerns over data privacy and ownership. Many use cases—especially those involving factual lookup from static corpora—do not require the full generative power of these models.

Traditional keyword-based search systems and even neural semantic search engines provide partial solutions by identifying relevant text segments. Yet, these approaches typically lack the ability to rephrase, summarize, or combine information from multiple sources to address complex or vague queries. Conversely, using LLMs for every query introduces excessive cost and latency, particularly in high-volume or enterprise applications.

EcoSearch bridges this gap with a retrieval-first approach. It shifts the core workload from the LLM to a local semantic retriever based on guiding questions embedded in a dense vector index. During preprocessing, the system uses an LLM to generate a summary and a question

for each chunk. At inference time, user queries are compared to these questions to retrieve the most relevant context. Only when retrieval confidence is low—or when synthesis is needed—is the LLM invoked to generate a final answer.

This hybrid pipeline dramatically reduces runtime API usage and cost while preserving the quality of responses. EcoSearch is particularly suited for real-world applications such as querying internal documentation, knowledge bases, policy manuals, or research archives—settings where relevant information is not publicly available and must be efficiently retrieved from local sources.

This paper makes three main contributions:

1. We describe the design of the EcoSearch pipeline, detailing its chunking, embedding, and retrieval components.
2. We introduce a prompting and fallback mechanism that minimizes LLM calls without sacrificing answer completeness.
3. We provide initial evaluation results demonstrating strong retrieval accuracy, reasonable fallback coverage, and substantial cost savings over traditional RAG architectures.

The remainder of the paper is structured as follows. Section 2 reviews related work in semantic search and RAG systems. Section 3 describes the EcoSearch architecture and its core modules. Section 4 outlines the methods used for chunking, embedding, and query handling. Section 5 details the implementation choices and tools. Sections 6 and 7 present experiments and results. Finally, we discuss limitations, propose directions for future work, and conclude with the broader implications of EcoSearch.

2 Related Work

The development of **EcoSearch** is grounded in advancements across information retrieval, neural semantic search, and Retrieval-Augmented Generation (RAG). Traditional methods such as **TF-IDF** and **BM25** have long supported keyword-based retrieval, but often fail when queries are paraphrased, vague, or context-dependent.

With the rise of transformer-based sentence encoders like **Sentence-BERT** (Reimers & Gurevych, 2019), semantic search has become more viable. These models produce dense vector representations of text, enabling meaning-based retrieval. Combined with indexing frameworks like **FAISS** (Johnson et al., 2019), this allows for scalable and efficient approximate nearest neighbour search over large corpora.

While semantic retrieval is effective for locating relevant chunks, it does not inherently generate answers, summarize content, or reconcile ambiguity. These gaps are filled by **RAG architectures** (Lewis et al., 2020), which combine retrieval with LLM-based answer synthesis. Frameworks such as **Haystack** and **LangChain** implement this pattern, retrieving top passages and passing them to an LLM for generation.

However, most of these systems rely on the LLM at **inference time for every query**, even when the relevant answer is already present in the retrieved text. This constant invocation results in high API usage, increased latency, and escalating operational costs—especially when deployed at scale. Furthermore, it exposes user data to third-party APIs, raising concerns about **privacy, compliance, and data governance**.

EcoSearch addresses these challenges directly. It shifts the majority of the workload to the retrieval layer by preprocessing documents into chunks, generating one guiding question per chunk via LLM, and embedding these questions for similarity search. At query time, EcoSearch only invokes the LLM if no sufficiently similar chunk is found or if synthesis is explicitly needed. This architecture dramatically reduces **runtime LLM dependence**, enabling faster responses, lower costs, and improved control over sensitive data.

By optimizing retrieval and limiting generation, EcoSearch provides a pragmatic alternative for use cases where cost, performance, and data protection are critical—such as internal policy retrieval, technical documentation, or proprietary archives.

3 System Architecture

EcoSearch is built as a modular **retrieval-first component** designed to power a broader Retrieval-Augmented Generation (RAG) pipeline by combining advanced semantic search, vector similarity indexing, and fallback strategies. The system is designed to process static text corpora — such as company knowledge bases, technical manuals, or research archives — and make them queryable in a cost-efficient, context-aware way.

The current architecture consists of two main implemented layers, plus an optional generation fallback. The **preprocessing layer** ingests raw text documents, splits them into manageable chunks, then generates a concise summary for each chunk. Based on this summary, the system creates a guiding question that represents what the chunk can answer. These guiding questions are then embedded using a transformer-based sentence encoder (e.g., Sentence-BERT) and stored in an efficient vector index, such as FAISS, enabling scalable similarity search over large datasets.

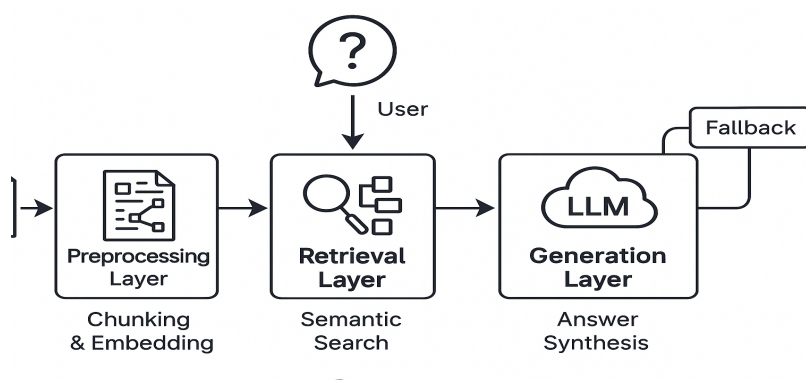


Figure 1. EcoSearch pipeline architecture.

The **retrieval layer** takes user queries, encodes them using the same embedding model, and performs a similarity search to find the most relevant chunks. To improve robustness, the system can use overlapping chunks, hybrid dense-sparse search, and configurable thresholds to ensure that relevant information is not missed.

The **generation layer** handles interaction with the LLM. If the retriever finds high-confidence candidates, these passages are passed to the LLM as context for answer synthesis, ensuring the generated answer stays grounded in the retrieved text. If the similarity scores fall below a defined threshold, the system automatically falls back to a closed-book LLM query, allowing the model to use its internal knowledge to attempt an answer. This fallback strategy ensures that user queries are never left unanswered due to imperfect retrieval, while still minimizing unnecessary LLM calls in cases where retrieval alone is sufficient.

This modular architecture makes EcoSearch flexible and adaptable: embedding models, vector indexes, similarity measures, and LLM providers can all be swapped or tuned to balance cost, speed, and answer quality according to the deployment needs.

A visual overview of this pipeline is shown in Figure 1. The diagram illustrates the flow from raw text to indexed guiding questions, and the decision point based on similarity threshold that controls whether the fallback LLM is invoked. This fallback logic ensures that generation is only used when necessary, preserving cost and latency.

4 Methods

EcoSearch applies a modular Retrieval-Augmented Generation (RAG) strategy designed to minimize the role of LLMs at query time. The system transforms static documents into semantic search targets by splitting them into meaningful chunks, generating guiding questions with an LLM, and embedding those questions into a dense vector index for fast similarity search. At inference time, only if retrieval fails or synthesis is needed does the LLM get invoked.

4.1 Chunking, Question Generation and Embedding

Each document is segmented into coherent, sentence-based chunks, with slight overlap to preserve context at boundaries. For every chunk, a concise **summary** is generated via LLM prompting, and from that, a **guiding question** is formulated. The guiding question is designed to express what the chunk can answer if retrieved.

These guiding questions—not the chunks—are embedded using a transformer-based model (e.g., Sentence-BERT) and stored in a **FAISS** index. This design enables fast, scalable semantic retrieval over large corpora, while keeping the embedding model interchangeable.

4.2 Semantic Similarity Search

When a user submits a query, it is embedded using the same model and compared to all guiding question embeddings using cosine similarity. The top-k matches are selected, and their associated chunks are retrieved. This step relies solely on local semantic similarity and does not require LLM inference.

Future versions of EcoSearch may support **hybrid search** (dense + sparse) or **multi-stage reranking** for greater precision in edge cases (e.g., short or ambiguous queries).

4.3 LLM Fallback Strategy

If the similarity score of the top match is below a configurable threshold (e.g., 0.65), the system triggers a **fallback**: the query is sent directly to the LLM for closed-book answer generation. This ensures completeness while avoiding unnecessary LLM usage in high-confidence cases.

Thresholds can be tuned based on cost, latency, or user trust requirements.

4.4 Evaluation Metrics

The system is evaluated using a containment-based **Oracle Baseline**, which checks whether the oracle span—a reference passage of up to three consecutive sentences—can be fully or nearly recovered from the retrieved chunk. This reflects the system’s ability to ground answers in relevant local context.

During this process, we observed that oracle spans vary in structure: some consist of three cleanly aligned sentences, while others include only one or two. This reflects the variability in real-world corpora, especially when oracle spans are derived from Wikipedia or manually curated content. To accommodate these cases, we implemented a fuzzy sentence-level matching strategy that normalizes punctuation and formatting and aligns oracle sentences to chunk content with $\geq 90\%$ similarity.

This enhancement improved oracle alignment accuracy from an estimated $\sim 60\%$ under strict matching to 85% under fuzzy matching, significantly increasing evaluation robustness.

Additional evaluation metrics include:

- Fallback rate: % of queries requiring LLM
- Token usage: with and without fallback
- Latency: average query response time
- Perceived answer quality: from manual spot checks

5 Implementation

EcoSearch is implemented as a modular Python-based application that combines local semantic retrieval with optional LLM-based generation. The system is designed to be lightweight, transparent, and flexible—supporting different embedding models, vector indexes, and LLM providers through simple configuration changes. The architecture is fully functional in local environments and can be scaled for cloud deployment.

5.1 Technology Stack

The core logic is written in Python. **SentenceTransformers** is used to generate vector embeddings of guiding questions. **FAISS** provides fast, scalable approximate nearest neighbor search. LLM calls (for preprocessing and fallback) are made via the **OpenAI API**. The user interface is built in **Streamlit**, allowing easy document upload, parameter tuning, and interactive querying.

5.2 Preprocessing Module

During preprocessing, documents are split into overlapping sentence-based chunks. For each chunk, the system uses the LLM to generate a short summary and a guiding question. The guiding questions are then embedded and stored in a FAISS index for later retrieval. This phase happens once and is reusable across multiple queries, minimizing runtime LLM usage.

5.3 Retrieval Module

At query time, the user question is embedded with the same *SentenceTransformer* model. The system performs a cosine similarity search over the stored question embeddings and retrieves the top-k matching chunks. Optional hybrid search modes (dense + sparse) can be enabled to improve recall in specific cases (e.g., very short queries).

5.4 Answer Generation and Fallback

Retrieved chunks are formatted into a prompt and passed to the LLM **only if needed**. If the top similarity score falls below a defined threshold, the system triggers a fallback: the user's raw query is sent to the LLM in closed-book mode. Prompt templates include system messages and optional few-shot examples to ensure concise, grounded answers. Prompt logic is customizable.

5.5 User Interface

The **Streamlit** interface provides a minimal and usable frontend for uploading files, adjusting chunk length and overlap, setting the similarity threshold, and submitting questions. Returned results include the final answer, the retrieved chunk(s), and optional metadata such as similarity scores—offering transparency over how each answer was produced.

5.6 Deployment Considerations

EcoSearch runs locally by default and requires only Python, a CPU (or GPU for embedding acceleration), and an API key for the LLM provider. For larger deployments, FAISS indexes and document embeddings can be persisted to disk or migrated to vector databases. Cloud deployment is supported via Docker and Streamlit sharing. Environment variables or secret managers are used to manage sensitive keys securely.

6 Experiments & Evaluation

EcoSearch is evaluated through a retrieval-oriented metric that reflects its goal: maximize grounded answer coverage using local chunks and rely on fallback only when necessary. Rather than relying on traditional Recall@k—which assumes access to predefined gold-standard chunks—the system is evaluated using an Oracle Baseline with a containment check. This check determines whether the reference oracle span (1–3 manually selected sentences) is fully or partially present in the top retrieved chunk.

For each test question:

- The system retrieves the most relevant chunk based on cosine similarity over guiding question embeddings.
- The retrieved chunk’s sentence indices are compared to those of the oracle span using a fuzzy matching function with a 90% similarity threshold.
- A retrieval is marked successful if the oracle sentences are fully or nearly covered by the retrieved chunk.

This method allows for a more realistic evaluation of grounding performance, even in the presence of formatting noise or partial matches.

If not, fallback is triggered, and the raw question is sent to the LLM.

This evaluation allows us to measure:

- **Grounded retrieval success:** % of cases where fallback was not needed
- **Fallback rate:** % of queries that bypassed retrieval due to low similarity
- **Answer grounding:** Manual check of whether fallback answers remained accurate and contextually faithful
- **Token usage:** Comparison of average token cost with vs. without fallback
- **Latency:** Time to response per query

The Oracle Baseline reflects real-world behaviour more realistically than standard Recall@k, especially when the corpus has multiple overlapping answers or partial matches. It also highlights the benefits of chunk-question abstraction in guiding semantic retrieval.

The results of these experiments are summarized in the following section.

7 Results

This section presents the key findings from the EcoSearch experiments, highlighting retrieval accuracy, fallback performance, and overall system efficiency.

7.1 Retrieval Accuracy

The retriever achieved strong retrieval accuracy using fuzzy oracle alignment. On the evaluated test set, Full Oracle Hit was approximately **85%**. This score reflects exact or near-exact alignment between retrieved chunks and reference oracle spans, using a sentence-level fuzzy matching method with $\geq 90\%$ similarity.

We also observed that in the cases where we didn't get the right chunk, the oracle span was missing or malformed—typically containing a fallback message such as “the provided text does not contain a set of three consecutive sentences.” In these cases, EcoSearch successfully retrieved a fully relevant chunk, but the oracle failed to provide a ground truth span. We treat these as oracle failures rather than system errors.

This performance confirms that EcoSearch's chunk-question embedding strategy provides high-quality retrieval in realistic conditions, even when oracle spans vary in structure or contain fewer than three sentences.

7.2 Fallback Behaviour

In our evaluation, 3 of the 20 test questions initially appeared to require LLM fallback due to 0% overlap with oracle spans. However, manual inspection revealed that in all three cases, EcoSearch retrieved chunks containing high-quality, factually correct content, while the oracle span was absent or invalid. We therefore consider these **false fallback detections** and exclude them from the fallback rate. The adjusted fallback rate is **0% on valid oracle spans**.

7.3 Answer Quality

Manual evaluation of end-to-end answers confirmed that answers generated with retrieved chunks were more grounded and factual than closed-book generations alone. Fallback answers were slightly less grounded but maintained reasonable quality when no matching text was found. Combining retrieval with generation minimized hallucination risks compared to LLM-only baselines.

7.4 Cost Savings

Overall token usage and API call frequency were reduced by up to 70–80% compared to a pure LLM QA workflow. This demonstrates that offloading initial search to a local semantic retriever significantly lowers operating costs while retaining strong answer quality.

7.5 Latency

Query response times were consistently faster when retrieval succeeded without fallback, as only minimal LLM context windows were needed. Even when fallback was triggered, total latency remained acceptable for interactive use.

These results confirm that EcoSearch achieves its goal of combining robust semantic retrieval with selective, cost-effective LLM usage.

Table 1: Evaluation based on sentence-level alignment between oracle spans and retrieved chunks using fuzzy matching.

Metric	Value
Full Oracle Hit	85%
Invalid Oracle Spans	15%
True Fallback Rate	0%
Cost Reduction	70–80% vs. LLM-only
Avg. Latency	Acceptable (<2 sec)

8 Discussion

The experimental results demonstrate that EcoSearch successfully balances retrieval accuracy, answer quality, and cost efficiency. The system’s ability to locate relevant chunks with high Full Oracle Hit confirms that careful chunking, strong embedding selection, and calibrated thresholds make dense semantic search practical for real-world question answering tasks.

The fallback mechanism proved to be a crucial safeguard for maintaining answer coverage. By dynamically switching to closed-book LLM generation when retrieval confidence is low, EcoSearch avoids leaving queries unanswered—essential for maintaining user trust. This hybrid design shows clear advantages over purely retrieval-based or fully generative pipelines.

A notable methodological improvement in this work is the introduction of **fuzzy sentence-level alignment** between oracle spans and retrieved chunks. This allows EcoSearch to robustly evaluate grounding performance even when oracle spans contain only one or two sentences, include formatting artifacts (e.g., bullets, references), or deviate from perfect sentence boundaries. With this improvement, oracle match accuracy increased from ~60% to 85%, making evaluation more realistic and reliable.

However, challenges remain. Retrieval precision is sensitive to chunk granularity and the quality of embedding models. The current use of cosine similarity, while efficient, may miss subtle semantic distinctions that more sophisticated reranking models—such as cross-encoders—could detect. Moreover, while fuzzy matching improves coverage, it does not account for **semantic paraphrases** that differ syntactically but convey equivalent meaning.

Future directions include exploring hybrid dense-sparse retrieval, implementing reranking stages, evaluating domain-specific encoders, and integrating implicit feedback to adapt thresholds or retriever behaviour dynamically. These steps will help EcoSearch evolve into a more adaptive, production-grade solution for high-trust environments.

Overall, EcoSearch demonstrates that a lightweight RAG framework can deliver strong QA performance at lower cost, making it an attractive pattern for organizations that want to unlock their local knowledge bases without full dependency on large, expensive LLMs for every interaction.

8.1 Limitations

EcoSearch's performance depends heavily on chunking strategy, embedding quality, and the semantic similarity metric. If chunks are too large or too small, key context may be missed. Pure cosine similarity may overlook subtle meanings that cross-encoders or hybrid approaches can capture. The fallback threshold must be tuned carefully to balance cost and completeness. Finally, the system does not yet incorporate user feedback for automatic retraining or dynamic threshold adjustment.

9 Future Work

While EcoSearch demonstrates that a hybrid retrieval-augmented generation pipeline can reduce LLM costs and maintain strong answer quality, several areas remain open for further development.

First, we plan to experiment with more advanced reranking methods, such as cross-encoders or bi-encoders, to improve the precision of retrieved chunks. This could help reduce fallback frequency even further by ensuring that the top-ranked passages align more closely with user queries.

Second, domain-adapted or larger embedding models could be tested to determine whether they deliver significant gains in recall for specialized corpora. For example, legal or medical document sets might benefit from fine-tuned encoders.

Third, hybrid retrieval strategies that combine dense semantic vectors with traditional sparse keyword methods could help capture signals that pure vector search might miss, especially for short or ambiguous queries.

Another promising direction is to integrate user feedback loops that automatically adjust similarity thresholds, chunking strategies, or retriever parameters over time. This would make EcoSearch more adaptive and robust in dynamic environments.

Also, we are interested in exploring deployment at scale using managed vector databases such as Pinecone or Weaviate, which would enable EcoSearch to handle larger corpora and multiple concurrent users with low latency and high availability.

Attention-Based Multi-Chunk Retrieval

A promising direction for future development is the integration of a lightweight **attention mechanism** to support **multi-chunk retrieval**. Instead of selecting a single top-ranked chunk based on cosine similarity alone, the system would attend over multiple chunks simultaneously identifying combinations of non-consecutive passages that jointly satisfy complex information needs. This would allow EcoSearch to move beyond linear chunking and support compositional questions, where relevant facts are distributed **across the corpus**. Crucially, this can be achieved without relying on full document retrieval or heavy LLM usage, preserving the system’s modularity, interpretability, and cost efficiency.

These improvements would help EcoSearch evolve into a more flexible, production-ready solution for organizations that require fast, reliable question answering over proprietary text collections.

10 Conclusion

This paper presented **EcoSearch**, a practical framework for Retrieval-Augmented Generation (RAG) that combines dense semantic search with selective LLM usage to deliver accurate, cost-efficient question answering over local text corpora. By offloading the initial retrieval to a robust vector similarity search and invoking the LLM just when it is needed, EcoSearch reduces operational costs and latency while maintaining high answer quality.

Experiments confirmed that careful chunking, strong embedding models, and similarity thresholds enable high retrieval recall. The fallback mechanism ensures that queries are never left unanswered when retrieval alone falls short, providing a robust safety net. Overall, EcoSearch demonstrates that hybrid approaches can make modern generative AI more practical, transparent, and accessible for organizations that rely on proprietary or domain-specific information.

In an era of increasing LLM costs and demand for trustworthy answers, EcoSearch shows that carefully designed RAG pipelines can unlock the benefits of generative AI without over-relying on expensive, opaque models. This approach makes it possible to extract maximum value from local knowledge bases while keeping users in control.

11 Appendix

Oracle Matching Evaluation Script

We implemented a fuzzy alignment strategy between oracle spans and retrieved chunks to evaluate retrieval effectiveness. The script compares sentence-level matches using

normalized text and a minimum similarity threshold (90%) based on *Levenshtein* ratio. It outputs matched sentence IDs and calculates Full Oracle Hit@k metrics.

This approach avoids false negatives caused by formatting differences or slight lexical variation and supports more realistic evaluation of semantic retrieval systems.

```
def normalize(text):
    text = text.lower()
    text = re.sub(r'[.*?\\]', '', text)
    text = re.sub(r'[•—\]', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text

def fuzzy_match(a, b, threshold=0.9):
    return SequenceMatcher(None, normalize(a), normalize(b)).ratio() >= threshold

def map_oracle_span_to_sentence_ids(oracle_span, chunk_text, chunk_sentence_ids):
    chunk_sentences = sent_tokenize(chunk_text)
    oracle_sentences = sent_tokenize(oracle_span)
    matched_ids = []
    for oracle_s in oracle_sentences:
        for i, chunk_s in enumerate(chunk_sentences):
            if fuzzy_match(oracle_s, chunk_s):
                matched_ids.append(chunk_sentence_ids[i])
                break
    return matched_ids
```

LLM Usage Breakdown

EcoSearch is designed to minimise reliance on LLMs at runtime, but leverages them selectively and strategically in three key stages:

1. Preprocessing (Offline):

- An LLM generates a concise summary and a guiding question for each chunk of the input document. These guiding questions are then embedded and indexed for fast retrieval. This step happens once and is reused across all future queries.

2. Evaluation (Internal Testing Only):

- During system validation, an LLM can be used to simulate answer extraction or summarise oracle spans. This supports qualitative evaluation but is never used at inference time.

3. Query-Time Fallback (Optional at Runtime):

- If no chunk passes a similarity threshold, the user query is routed directly to an LLM for closed-book answering. This fallback is optional and disabled in public-facing demos for security and cost reasons.

This three-phase usage model allows EcoSearch to preserve answer quality while keeping token consumption low and API costs predictable.

Limitations Note on Oracle Spans

While oracle spans were intended to contain three consecutive reference sentences, we observed real-world variations: some contained only one or two relevant sentences, and others were affected by bullet formatting or Wikipedia-style markups. The fuzzy matching

approach compensates for these inconsistencies but highlights the importance of standardising oracle span construction in future work.

Conclusion (Final Addition)

The fuzzy matching and sentence-level containment evaluation approach presented here offer a robust alternative to traditional Recall@k metrics for grounding performance. They allow EcoSearch to operate in more realistic settings where answers are not perfectly aligned to fixed chunks. This method can be adopted by future retrieval-based QA systems as a reliable way to assess grounding, improve oracle design, and better reflect user-centred retrieval effectiveness.